Cyberscope

A TAC Security Company

Audit Report

Wake Token

April 2026

Repository https://github.com/paradigmajs/WAKE_TOKEN/tree/main/contracts

Commit 8520a6f3bef3736984c473bad63285f4f41bd267

Audited by © cyberscope

Table of Contents

Table of Contents **1**
Risk Classification **3**
Review **4**
Audit Updates 4
Source Files 4
Overview **6**
WAKEToken.sol 6
WakeTimelockController.sol 6
WakeVestingMath.sol 6
WakeBeneficiaryVestingVault.sol 7
WakeBoundEmissionVault.sol 7
WakePresaleMerkleVesting.sol 8
WakeStaking.sol 8
WakeLiquidityVault.sol 9
WakeTimelockVault.sol 9
Diagnostics **10**
Findings Breakdown **12**
IVC - Incorrect Vesting Calculation 13
Description 13
Recommendation 14
TSI - Tokens Sufficiency Insurance 15
Description 15
Recommendation 16
ODCL - Order Dependent Claim Logic 17
Description 17
Recommendation 17
CR - Code Repetition 18
Description 18
Recommendation 20
CCR - Contract Centralization Risk 21
Description 21
Recommendation 22
DDP - Decimal Division Precision 23
Description 23
Recommendation 24
EDBA - External Dependency Blocks Actions 25
Description 25
Recommendation 26
MPC - Merkle Proof Centralization 27

Description 27
Recommendation 28
MC - Missing Check 29
Description 29
Recommendation 30
OCED - Owner Controls Emission Destination 31
Description 31
Recommendation 32
PTAI - Potential Transfer Amount Inconsistency 33
Description 33
Recommendation 34
ROF - Redundant Ownable Functionality 35
Description 35
Recommendation 35
RVA - Redundant Variable Assignment 36
Description 36
Recommendation 36
URR - Unstake Request Reset 37
Description 37
Recommendation 37
L09 - Dead Code Elimination 38
Description 38
Recommendation 39
L17 - Usage of Solidity Assembly 40
Description 40
Recommendation 41
L19 - Stable Compiler Version 42
Description 42
Recommendation 43
Functions Analysis **44**
Inheritance Graph **50**
Flow Graph **51**
Summary **52**
Disclaimer **53**
About Cyberscope **54**

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	26 Mar 2026 https://github.com/cyberscope-io/audits/blob/main/wake/v1/audit.pdf
Corrected Phase 2	02 Apr 2026

Source Files

Filename	SHA256
WakeVestingMath.sol	ee112aad5592853465797c0209b3550a984693530cb53e775ab69d56bd9fa7f4
WakeTimelockVault.sol	ef1d1c6624a995272d1e3cf7c6f2ccb942108d07aa950037adf78ab239c8566f
WakeTimelockController.sol	96c57caa748032cb80e264802ff32779173e37bf8df1f65bf499fae1f9c7e5d6
WakeStaking.sol	9a0fd4c2c1135f4d9b8394898e48d5abedc2e305a53ca1d119e93526cecafc8e
WakePresaleMerkleVesting.sol	44893c174375b7613b872497f2563dbfbac4e73554ff23be8f18fa489a159c2b
WakeLiquidityVault.sol	45565de29c56a0558eb2c41b2b87e6ab0b3b32cf54240a1596d4c5ded7e6ab93
WakeBoundEmissionVault.sol	6a2cbe97c7ac6dc68d1df3ee3687a27f838dc93654c4167b307dcee6b8a1356e
WakeBeneficiaryVestingVault.sol	9fbd5748b4d715b864459df322cf05086eb8d1238965e178950399f11a207e5c

WAKEToken.sol	70c3e92e55a242d7efb10c35f7f02210ab4cda3a0db6d6e63fa881ea2dfa4a93

Overview

WAKEToken.sol

WAKE Token is a standard ERC-20 token built on OpenZeppelin's `ERC20` implementation with a fixed supply of 1,575,137,505 tokens minted entirely to a designated `initialHolder` address at construction. The contract enforces a zero-address check on the initial holder via a custom `InvalidInitialHolder` error. No mint, burn, pause, blacklist, tax, or administrative functions exist beyond the constructor — the token is fully immutable after deployment with no owner privileges and no supply modification capability.

WakeTimelockController.sol

WakeTimelockController is a thin wrapper around OpenZeppelin's `TimelockController` governance primitive. The constructor accepts a minimum delay, arrays of proposer and executor addresses, and an admin address, passing them directly to the parent implementation. No custom logic, overrides, or additional state variables are introduced. The contract inherits the full OpenZeppelin timelock functionality including proposal scheduling, execution delays, role-based access control, and cancellation capabilities.

WakeVestingMath.sol

WakeVestingMath is a stateless Solidity library providing a shared monthly vesting calculation consumed by multiple vault contracts in the ecosystem. The `vestedAmount()` function accepts a total allocation, TGE timestamp, cliff duration, vesting duration, initial unlock percentage in basis points, and a query timestamp, returning the vested token amount at that point in time. The calculation releases an initial tranche at TGE based on `initialUnlockBps`, enforces a cliff period during which no additional tokens vest, and then unlocks the remaining allocation in monthly increments using 30-day

periods. The month counter starts at `+1` after the cliff, and `totalMonths` is derived via integer division of the post-cliff duration by 30 days.

WakeBeneficiaryVestingVault.sol

WakeBeneficiaryVestingVault is a single-beneficiary token vesting contract that locks a fixed allocation and releases it according to a configurable schedule powered by the shared `WakeVestingMath` library. All schedule parameters — token address, beneficiary, total allocation, TGE timestamp, cliff duration, vesting duration, and initial unlock basis points — are set as immutables at construction with validation that the cliff does not exceed the vesting duration and the initial unlock does not exceed 100%. The `release()` function is permissionless and can be called by any address, but tokens are always transferred exclusively to the immutable `beneficiary`. A `released` counter tracks cumulative distributions to prevent double-claiming. The contract inherits `Ownable` but defines no `onlyOwner` functions.

WakeBoundEmissionVault.sol

WakeBoundEmissionVault is a linear token emission vault designed to feed staking rewards over a configured duration. The vault holds a `totalAllocation` of tokens and releases them proportionally based on elapsed time since `startTimestamp` over the `emissionDuration`, using a straightforward `(totalAllocation * elapsed) / emissionDuration` formula. Only the designated `controller` address can call `releaseAvailable()` to pull vested tokens, and the `released` counter ensures no double-withdrawal. The owner retains the ability to change the `controller` address at any time via `setController()`, which emits a `ControllerUpdated` event. The constructor validates against zero addresses and a zero emission duration.

WakePresaleMerkleVesting.sol

WakePresaleMerkleVesting is a Merkle-tree-based vesting contract that distributes tokens to presale participants according to a shared monthly schedule powered by the `WakeVestingMath` library. Each participant's total allocation is encoded in the Merkle leaf alongside their address, and claims are verified against a stored `merkleRoot` using OpenZeppelin's `MerkleProof`. The root can be updated by the owner via `setMerkleRoot()` until permanently locked through `freezeRoot()`. Users claim vested tokens through `claim()`, direct them to another address via `claimTo()`, or claim and immediately stake through `claimAndStake()` which transfers tokens to the contract, approves the provided staking address, and calls `stakeFor()` on the target. A per-address `claimed` mapping tracks cumulative withdrawals to enforce incremental vesting. All vesting parameters — TGE timestamp, cliff duration, vesting duration, and initial unlock basis points — are set as immutables at construction.

WakeStaking.sol

WakeStaking is a single-asset staking contract that distributes token rewards pulled from an external `WakeBoundEmissionVault` using an accumulator-based reward distribution model. Users stake WAKE tokens via `stake()` or `stakeFor()`, and rewards are tracked through an `accRewardPerShare` accumulator scaled by `ACC_PRECISION` (1e24). Every state-changing operation — staking, unstaking, claiming, and compounding — first calls `_syncRewards()` which pulls available emissions from the vault and distributes them proportionally across all active stakers. If no tokens are staked when emissions arrive, they are recorded as `undistributedRewards` which the owner can later redirect via `releaseUndistributedRewards()`. Unstaking follows a two-step process: `requestUnstake()` moves tokens from active stake to a pending withdrawal with an `unbondingPeriod` cooldown, and `withdrawUnstaked()` transfers the matured tokens back to the user. Users can also `compoundRewards()` to add unclaimed rewards directly to their active stake without an external transfer. The contract uses OpenZeppelin's `ReentrancyGuard` on all external functions and `SafeERC20` for all token operations.

WakeLiquidityVault.sol

WakeLiquidityVault is a time-locked token holding contract that prevents the release of tokens before a configured `unlockTimestamp`. The owner can call `releaseTo()` to transfer any amount of the held token to a specified recipient, but only after the unlock timestamp has passed. The constructor sets the token address and unlock timestamp as immutables with zero-address validation on both the owner and token parameters. The contract uses OpenZeppelin's `SafeERC20` for token transfers and emits a `Released` event on each withdrawal.

WakeTimelockVault.sol

WakeTimelockVault is a time-locked token holding contract identical in implementation to `WakeLiquidityVault`. The owner can call `releaseTo()` to transfer any amount of the held token to a specified recipient after the configured `unlockTimestamp`. The constructor sets the token address and unlock timestamp as immutables with zero-address validation. The contract uses OpenZeppelin's `SafeERC20` for token transfers and emits a `Released` event on each withdrawal.

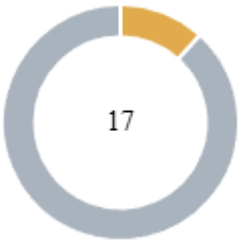
Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Medium	IVC	Incorrect Vesting Calculation	Unresolved
Medium	TSI	Tokens Sufficiency Insurance	Unresolved
Minor / Informative	ODCL	Order Dependent Claim Logic	Unresolved
Minor / Informative	CR	Code Repetition	Unresolved
Minor / Informative	CCR	Contract Centralization Risk	Unresolved
Minor / Informative	DDP	Decimal Division Precision	Unresolved
Minor / Informative	EDBA	External Dependency Blocks Actions	Unresolved
Minor / Informative	MPC	Merkle Proof Centralization	Unresolved
Minor / Informative	MC	Missing Check	Unresolved
Minor / Informative	OCED	Owner Controls Emission Destination	Unresolved
Minor / Informative	PTAI	Potential Transfer Amount Inconsistency	Unresolved
Minor / Informative	ROF	Redundant Ownable Functionality	Unresolved
Minor / Informative	RVA	Redundant Variable Assignment	Unresolved
Minor / Informative	URR	Unstake Request Reset	Unresolved

●	L09	Dead Code Elimination	Unresolved
●	L17	Usage of Solidity Assembly	Unresolved
●	L19	Stable Compiler Version	Unresolved

Findings Breakdown



Critical	0
Medium	2
Minor / Informative	15

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	2	0	0	0
Minor / Informative	15	0	0	0

IVC - Incorrect Vesting Calculation

Criticality	Medium
Location	WakeVestingMath.sol#L44
Status	Unresolved

Description

The shared vesting library calculates monthly vesting using a rounded month counter that introduces two compounding inaccuracies. First, `monthsElapsed` adds `+1` to the elapsed month count, causing the first vesting tranche to unlock immediately at the cliff end without any time having passed in the first vesting period. Second, `totalMonths` is derived via integer division of `postCliffDuration / MONTH`, which truncates any remainder. The combination of the immediate first tranche and integer division truncation causes full vesting to be reached before the configured vesting end. For durations shorter than 30 days, `totalMonths` becomes zero, triggering an immediate full vest. Since the library is used by both `WakeBeneficiaryVestingVault` and `WakePresaleMerkleVesting`, the issue propagates across multiple allocation streams.

```shell
uint256 elapsedAfterCliff = timestamp - cliffEnd;
uint256 monthsElapsed = (elapsedAfterCliff /
MONTH) + 1;

uint256 totalMonths = postCliffDuration / MONTH;

if (totalMonths == 0 || monthsElapsed >=
totalMonths) {
    return totalAllocation;
}

uint256 vestedRemaining = (remaining *
monthsElapsed) / totalMonths;
```

```
return initial + vestedRemaining;
```

Recommendation

The team is advised to review the vesting formula to ensure that vesting begins only after the intended interval has elapsed and that the full allocation is not released before the configured vesting end. The implementation should consistently handle durations that are not exact multiples of 30 days and avoid premature completion of the vesting schedule.

TSI - Tokens Sufficiency Insurance

Criticality	Medium
Location	WakeBoundEmissionVault.sol#L26 WakeStaking.sol#L177
Status	Unresolved

Description

The `WakeBoundEmissionVault` calculates token emissions based on a configured `totalAllocation` but does not include a mechanism to verify or enforce that the corresponding tokens have been deposited. The constructor sets the allocation amount without transferring tokens into the contract, and no dedicated funding function exists. If the vault is partially funded or not funded at all, the `releaseAvailable()` function will revert on `safeTransfer`, which in turn causes `_syncRewards()` in `WakeStaking` to revert, blocking all staking operations including stake, unstake requests, reward claims, and compounding. Furthermore, as described in finding IVC, the vesting calculation may complete before the configured end date. If the vault funding is calibrated to the configured schedule, the accelerated release may deplete the available balance before the expected end, producing the same blocking effect during the final emission period.

Shell

```
constructor(...) Ownable(owner_) {
    token = token_;
    totalAllocation = totalAllocation_;
    startTimestamp = startTimestamp_;
    emissionDuration = emissionDuration_;
    controller = controller_;
}

function releaseAvailable() external returns
(uint256 amount) {
    if (msg.sender != controller) revert
```

```
NotController();
    amount = availableToRelease();
    if (amount == 0) return 0;
    released += amount;
    token.safeTransfer(controller, amount);
}
```

Recommendation

The team is advised to ensure that vault contracts are adequately funded before the emission schedule begins. The implementation could include a balance verification mechanism that confirms the contract holds sufficient tokens to cover the configured allocation, preventing operational disruptions caused by insufficient funding.

ODCL - Order Dependent Claim Logic

Criticality	Minor / Informative
Location	WakePresaleMerkleVesting.sol#L33,120
Status	Unresolved

Description

The contract tracks cumulative claims using a single `claimed[account]` accumulator per address. The Merkle proof verification does not enforce that each address appears with only one leaf in the tree. If the off-chain tree includes multiple leaves for the same address with different allocation amounts, the shared accumulator causes order-dependent claiming — a larger allocation claimed first can permanently block a smaller one, and a smaller allocation claimed first reduces the claimable amount from the larger one.

```shell
mapping(address => uint256) public claimed;

claimed[account] += amount;
```

Recommendation

The team is advised to ensure that the off-chain Merkle tree construction enforces a single leaf per address, or to implement per-leaf claim tracking that supports multiple allocations independently.

CR - Code Repetition

Criticality	Minor / Informative
Location	WakeLiquidityVault.sol#L8 WakeTimelockVault.sol#L8
Status	Unresolved

Description

The `WakeLiquidityVault` and `WakeTimelockVault` contracts share an identical implementation the same state variables, constructor logic, validation checks, and `releaseTo()` function differing only in the contract name. This duplication increases the maintenance surface and introduces the risk of inconsistency if future modifications are applied to one contract but not the other. The shared logic could be consolidated into a single abstract base contract, with each vault inheriting from it to preserve distinct contract identities for deployment and verification purposes.

Shell

```
contract WakeLiquidityVault is Ownable {
    using SafeERC20 for IERC20;

    error InvalidAddress();
    error Locked();

    IERC20 public immutable token;
    uint64 public immutable unlockTimestamp;

    event Released(address indexed to, uint256 amount);

    constructor(address owner_, IERC20 token_, uint64
unlockTimestamp_) Ownable(owner_) {
        if (owner_ == address(0) || address(token_) ==
address(0)) revert InvalidAddress();
        token = token_;
        unlockTimestamp = unlockTimestamp_;
    }

    function releaseTo(address to, uint256 amount) external
onlyOwner {
        if (to == address(0)) revert InvalidAddress();
        if (block.timestamp < unlockTimestamp) revert
Locked();
        token.safeTransfer(to, amount);
        emit Released(to, amount);
    }
}


contract WakeTimelockVault is Ownable {
    using SafeERC20 for IERC20;

    error InvalidAddress();
    error Locked();

    IERC20 public immutable token;
    uint64 public immutable unlockTimestamp;
```

```
    event Released(address indexed to, uint256 amount);

    constructor(address owner_, IERC20 token_, uint64
unlockTimestamp_) Ownable(owner_) {
        if (owner_ == address(0) || address(token_) ==
address(0)) revert InvalidAddress();
        token = token_;
        unlockTimestamp = unlockTimestamp_;
    }

    function releaseTo(address to, uint256 amount) external
onlyOwner {
        if (to == address(0)) revert InvalidAddress();
        if (block.timestamp < unlockTimestamp) revert
Locked();
        token.safeTransfer(to, amount);
        emit Released(to, amount);
    }

}
```

Recommendation

The team is advised to extract the common implementation into an abstract base contract. Each vault can then inherit from the shared base, reducing code duplication while maintaining separate contract identities. This approach improves maintainability and ensures that any future changes are applied consistently across both vaults.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	WakeBoundEmissionVault.sol#L44 WakeLiquidityVault.sol#L25 WakeTimelockVault.sol#L25 WakePresaleMerkleVesting.sol#L59 WakePresaleMerkleVesting.sol#L66
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Shell

```
function setController(address controller_)
external onlyOwner {

function releaseTo(address to, uint256 amount)
external onlyOwner {

function releaseTo(address to, uint256 amount)
external onlyOwner {

function setMerkleRoot(bytes32 newRoot) external
onlyOwner {


function freezeRoot() external onlyOwner {
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

DDP - Decimal Division Precision

Criticality	Minor / Informative
Location	WakeVestingMath.sol#L52 WakeBoundEmissionVault.sol#L59
Status	Unresolved

Description

Division of decimal (fixed point) numbers can result in rounding errors due to the way that division is implemented in Solidity. Thus, it may produce issues with precise calculations with decimal numbers.

Solidity represents decimal numbers as integers, with the decimal point implied by the number of decimal places specified in the type (e.g. decimal with 18 decimal places). When a division is performed with decimal numbers, the result is also represented as an integer, with the decimal point implied by the number of decimal places in the type. This can lead to rounding errors, as the result may not be able to be accurately represented as an integer with the specified number of decimal places.

Hence, the splitted shares will not have the exact precision and some funds may not be calculated as expected.

```
Shell

uint256 vestedRemaining = (remaining *
monthsElapsed) / totalMonths;


return (totalAllocation * elapsed) /
uint256(emissionDuration);
```

Recommendation

The team is advised to take into consideration the rounding results that are produced from the solidity calculations. The contract could calculate the subtraction of the divided funds in the last calculation in order to avoid the division rounding issue.

EDBA - External Dependency Blocks Actions

Criticality	Minor / Informative
Location	WakeStaking.sol#L64,82,177
Status	Unresolved

Description

The `_syncRewards()` function performs an external call to `emissionVault.releaseAvailable()` on every state-changing operation. The functions `stake()`, `stakeFor()`, `requestUnstake()`, `claimRewards()`, and `compoundRewards()` all invoke _syncRewards() as a prerequisite step. If the emission vault becomes non-functional, is paused, or its controller is changed to a different address, the `releaseAvailable()` call will revert, blocking all staking operations including the ability to initiate unstaking.

Shell

```
function _syncRewards() internal returns (uint256
amount) {
    amount = emissionVault.releaseAvailable();
    if (amount == 0) return 0;
    rewardsPulled += amount;
    ...
}

function requestUnstake(uint256 amount) external
nonReentrant {
    if (amount == 0) revert InvalidAmount();
    _syncRewards();
    ...
}

function withdrawUnstaked() external nonReentrant
{
```

```
        UserInfo storage user = users[msg.sender];
        uint256 amount = user.pendingWithdrawal;
        ...
    }
```

Recommendation

The team is advised to implement a fallback mechanism that allows critical user operations, particularly unstake requests, to proceed even when the external emission vault is unavailable.

MPC - Merkle Proof Centralization

Criticality	Minor / Informative
Location	WakePresaleMerkleVesting.sol#L59
Status	Unresolved

Description

The contract uses a Merkle Proof mechanism in order to define many applicable addresses. The verification process is based on an off-chain configuration. The contract owner is responsible for updating the in-chain “Merkle Root” in order to validate correctly the provided message.

```shell
function setMerkleRoot(bytes32 newRoot) external
onlyOwner {
    if (rootIsFrozen) revert RootFrozen();
    if (newRoot == bytes32(0)) revert
InvalidRoot();
    merkleRoot = newRoot;
    emit MerkleRootUpdated(newRoot);
}
```

Recommendation

We state that the Merkle Proof algorithm is required for proper protocol operations and gas consumption decrease. Thus, we emphasize that the Merkle proof algorithm is based on an off-chain mechanism. Any off-chain mechanism could potentially be compromised and affect the on-chain state unexpectedly. The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

MC - Missing Check

Criticality	Minor / Informative
Location	WakeStaking.sol#L53 WakeBoundEmissionVault.sol#L38,39 WakePresaleMerkleVesting.sol#L52 WakeBeneficiaryVestingVault.sol#L42,43 WakeLiquidityVault.sol#L22 WakeTimelockVault.sol#L22
Status	Unresolved

Description

The contract constructors accept configuration parameters without adequate boundary validation. Specifically, `unbondingPeriod` is not bounded by a maximum value, `totalAllocation` and `unlockTimestamp` are not checked for zero, and `startTimestamp` , `tgeTimestamp` , and `unlockTimestamp` are not validated against the current block timestamp to prevent initialization in the past.

Shell

```
unbondingPeriod = unbondingPeriod_;

totalAllocation = totalAllocation_;
startTimestamp = startTimestamp_;

tgeTimestamp = tgeTimestamp_;

totalAllocation = totalAllocation_;
tgeTimestamp = tgeTimestamp_;

unlockTimestamp = unlockTimestamp_;


unlockTimestamp = unlockTimestamp_;
```

Recommendation

The team is advised to properly check the variables according to the required specifications.

OCED - Owner Controls Emission Destination

Criticality	Minor / Informative
Location	WakeStaking.sol#L149
Status	Unresolved

Description

The contract accumulates released staking emissions as `undistributedRewards` whenever rewards are synced while no users are staking. The owner can later transfer these accumulated tokens to any arbitrary address through the `releaseUndistributedRewards()` function. While the accounting is properly maintained, this introduces an owner-directed diversion path for tokens originally allocated to the staking emission schedule.

```shell
function releaseUndistributedRewards(address to, uint256 amount) external onlyOwner nonReentrant {
    if (to == address(0)) revert InvalidAddress();
    if (amount == 0) revert InvalidAmount();
    if (amount > undistributedRewards) revert AmountExceedsUndistributed();

    undistributedRewards -= amount;
    token.safeTransfer(to, amount);

    emit UndistributedRewardsReleased(to, amount);
}
```

Recommendation

The team is advised to ensure that undistributed staking emissions remain dedicated to the staking reward system. A redistribution mechanism aligned with the intended tokenomics would reduce owner dependency over emission-allocated funds.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Minor / Informative
Location	WakeStaking.sol#L181
Status	Unresolved

Description

The `_syncRewards()` function uses the value returned by `emissionVault.releaseAvailable()` to update internal reward accounting. According to the ERC-20 specification, the actual amount received by a contract during a token transfer could potentially be less than the expected amount due to fees or taxes applied on transfer. The contract records this value directly into `rewardsPulled` and uses it to update `accRewardPerShare` without verifying the actual received balance. This may produce inconsistency between the expected and the actual reward distribution.

Shell

```
function _syncRewards() internal returns (uint256
amount) {
    amount = emissionVault.releaseAvailable();
    if (amount == 0) return 0;

    rewardsPulled += amount;

    accRewardPerShare += (amount * ACC_PRECISION)
/ totalStaked;
}
```

Recommendation

The team is advised to verify the actual token amount received by comparing the contract's balance before and after the transfer, rather than relying on the returned value. This ensures accurate reward accounting regardless of the token's transfer behavior.

ROF - Redundant Ownable Functionality

Criticality	Minor / Informative
Location	WakeBeneficiaryVestingVault.sol#L9
Status	Unresolved

Description

The contract inherits from the Ownable abstract contract to define an owner. In smart contracts, an owner typically has elevated privileges to execute administrative functions. However, in this case, while the contract defines an owner, it does not include any administrative functionalities. Therefore, the inheritance of Ownable is redundant.

```
Shell

contract WakeBeneficiaryVestingVault is Ownable {
```

Recommendation

Eliminating redundancies will reduce code size and enhance readability. By removing the unnecessary inheritance, the contract becomes more efficient and aids in future maintainability.

RVA - Redundant Variable Assignment

Criticality	Minor / Informative
Location	WakeStaking.sol#L200
Status	Unresolved

Description

The `_accrueUser()` function concludes by assigning `user.rewardDebt` based on the current `activeStake` and `accRewardPerShare`. However, every function that calls `_accrueUser()` subsequently modifies `activeStake` and reassigns `rewardDebt` with the updated value, overwriting the assignment made within `_accrueUser()`. This results in an unnecessary storage write on every call, increasing gas costs without contributing to the contract's logic.

```
Shell


user.rewardDebt = (user.activeStake *
accRewardPerShare) / ACC_PRECISION;
```

Recommendation

To improve efficiency and readability, redundant variable assignments should be removed. Variables should only be declared and assigned when they serve a functional purpose in computations or state modifications. Instead of creating unnecessary copies, values should be used directly in operations where applicable. Any unused or duplicate variables should be eliminated to minimize memory allocation and reduce gas consumption. By optimizing variable usage, the contract will become leaner, more cost-effective, and easier to understand, enhancing both security and maintainability.

URR - Unstake Request Reset

Criticality	Minor / Informative
Location	WakeStaking.sol#L76
Status	Unresolved

Description

The `requestUnstake()` function uses a single `pendingWithdrawal` accumulator and a single `unstakeAvailableAt` timestamp per user. Each call overwrites the `unstakeAvailableAt` value with a fresh timestamp, while the unstaked amount is accumulated into `pendingWithdrawal`. As a result, if a user initiates a second unstake request before withdrawing a matured previous request, the timer resets for the entire pending balance — including tokens that had already completed the unbonding period. This leads to temporary re-locking of already-matured funds.

```
Shell
user.pendingWithdrawal += amount;
user.unstakeAvailableAt = uint64(block.timestamp +
unbondingPeriod);
```

Recommendation

The team is advised to consider implementing a mechanism that prevents the unbonding timer from resetting for tokens that have already completed their unbonding period, ensuring that users do not experience unexpected delays when withdrawing matured funds.

L09 - Dead Code Elimination

Criticality	Minor / Informative
Location	WAKEToken.sol#L130,515 WakeTimelockVault.sol#L497 WakeTimelockController.sol#L129,340 WakeStaking.sol#L756 WakePresaleMerkleVesting.sol#L1035 WakeLiquidityVault.sol#L497 WakeBoundEmissionVault.sol#L497 WakeBeneficiaryVestingVault.sol#L497
Status	Unresolved

Description

In Solidity, dead code is code that is written in the contract, but is never executed or reached during normal contract execution. Dead code can occur for a variety of reasons, such as:

- Conditional statements that are always false.
- Functions that are never called.
- Unreachable code (e.g., code that follows a return statement).

Dead code can make a contract more difficult to understand and maintain, and can also increase the size of the contract and the cost of deploying and interacting with it.

```
Shell
function _contextSuffixLength() internal view virtual
returns (uint256) {
        return 0;
    }
on _burn(address account, uint256 value) internal {
        if (account == address(0)) {
            revert ERC20InvalidSender(address(0));
        _update(account, address(0), value);
```

Recommendation

To avoid creating dead code, it's important to carefully consider the logic and flow of the contract and to remove any code that is not needed or that is never executed. This can help improve the clarity and efficiency of the contract.

L17 - Usage of Solidity Assembly

Criticality	Minor / Informative
Location	WakeTimelockVault.sol#L369,411,448 WakeTimelockController.sol#L580,603,612,626,635,649,658,665,675,683 WakeStaking.sol#L369,411,448,539,548,557,566,575,584,593,602,611 WakePresaleMerkleVesting.sol#L369,411,448,498 WakeLiquidityVault.sol#L369,411,448 WakeBoundEmissionVault.sol#L369,411,448 WakeBeneficiaryVestingVault.sol#L369,411,448
Status	Unresolved

Description

Using assembly can be useful for optimizing code, but it can also be error-prone. It's important to carefully test and debug assembly code to ensure that it is correct and does not contain any errors.

Some common types of errors that can occur when using assembly in Solidity include Syntax, Type, Out-of-bounds, Stack, and Revert.

```
Shell
assembly ("memory-safe") {
            let fmp := mload(0x40)
            mstore(0x00, selector)
            mstore(0x04, and(to, shr(96, not(0))))
            mstore(0x24, value)
            success := call(gas(), token, 0, 0x00, 0x44,
0x00, 0x20)
...
                                success := and(success,
and(iszero(returndatasize()), gt(extcodesize(token), 0)))
            }
            mstore(0x40, fmp)
        }
```

Recommendation

It is recommended to use assembly sparingly and only when necessary, as it can be difficult to read and understand compared to Solidity code.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	WakeVestingMath.sol#L2 WakeTimelockVault.sol#L2 WakeTimelockController.sol#L2 WakeStaking.sol#L2 WakePresaleMerkleVesting.sol#L2 WakeLiquidityVault.sol#L2 WakeBoundEmissionVault.sol#L2 WakeBeneficiaryVestingVault.sol#L2 WAKEToken.sol#L2
Status	Unresolved

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```shell
pragma solidity ^0.8.24;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
WakeTimelockVault	Implementation	Ownable		
		Public	✓	Ownable
	releaseTo	External	✓	onlyOwner
AccessControl	Implementation	Context, IAccessControl, ERC165		
	supportsInterface	Public		-
	hasRole	Public		-
	_checkRole	Internal		
	_checkRole	Internal		
	getRoleAdmin	Public		-
	grantRole	Public	✓	onlyRole
	revokeRole	Public	✓	onlyRole
	renounceRole	Public	✓	-
	_setRoleAdmin	Internal	✓	
	_grantRole	Internal	✓	
	_revokeRole	Internal	✓	
ERC1155Holder	Implementation	ERC165, IERC1155Receiver		

	supportsInterface	Public		-
	onERC1155Received	Public	✓	-
	onERC1155BatchReceived	Public	✓	-
TimelockController	Implementation	AccessControl, ERC721Holder, ERC1155Holder		
		Public	✓	-
		External	Payable	-
	supportsInterface	Public		-
	isOperation	Public		-
	isOperationPending	Public		-
	isOperationReady	Public		-
	isOperationDone	Public		-
	getTimestamp	Public		-
	getOperationState	Public		-
	getMinDelay	Public		-
	hashOperation	Public		-
	hashOperationBatch	Public		-
	schedule	Public	✓	onlyRole
	scheduleBatch	Public	✓	onlyRole
	_schedule	Private	✓	
	cancel	Public	✓	onlyRole
	execute	Public	Payable	onlyRoleOrOpenRole

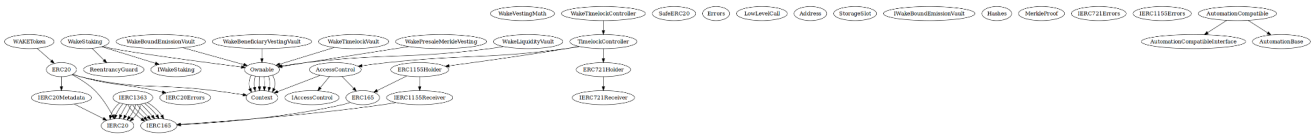
	executeBatch	Public	Payable	onlyRoleOrOpenRole
	_execute	Internal	✓	
	_beforeCall	Private		
	_afterCall	Private	✓	
	updateDelay	Public	✓	-
	_encodeStateBitmap	Internal		
WakeTimelockController	Implementation	TimelockController		
		Public	✓	TimelockController
WakeStaking	Implementation	Ownable, ReentrancyGuard, IWakeStaking		
		Public	✓	Ownable
	stake	External	✓	nonReentrant
	stakeFor	External	✓	nonReentrant
	requestUnstake	External	✓	nonReentrant
	withdrawUnstaked	External	✓	nonReentrant
	claimRewards	External	✓	nonReentrant
	compoundRewards	External	✓	nonReentrant
	pendingRewards	External		-
	effectiveStakeOf	External		-
	syncRewards	External	✓	nonReentrant
	releaseUndistributedRewards	External	✓	onlyOwner nonReentrant

	_stakeFrom	Internal	✓	
	_syncRewards	Internal	✓	
	_accrueUser	Internal	✓	
WakePresaleMerkleVesting	Implementation	Ownable		
		Public	✓	Ownable
	setMerkleRoot	External	✓	onlyOwner
	freezeRoot	External	✓	onlyOwner
	vestedAmount	Public		-
	claimable	Public		-
	claim	External	✓	-
	claimTo	External	✓	-
	claimAndStake	External	✓	-
	leaf	Public		-
	_claim	Internal	✓	
	_verify	Internal		
WakeLiquidityVault	Implementation	Ownable		
		Public	✓	Ownable
	releaseTo	External	✓	onlyOwner
	transferFrom	External	✓	-
WakeBoundEmissionVault	Implementation	Ownable		

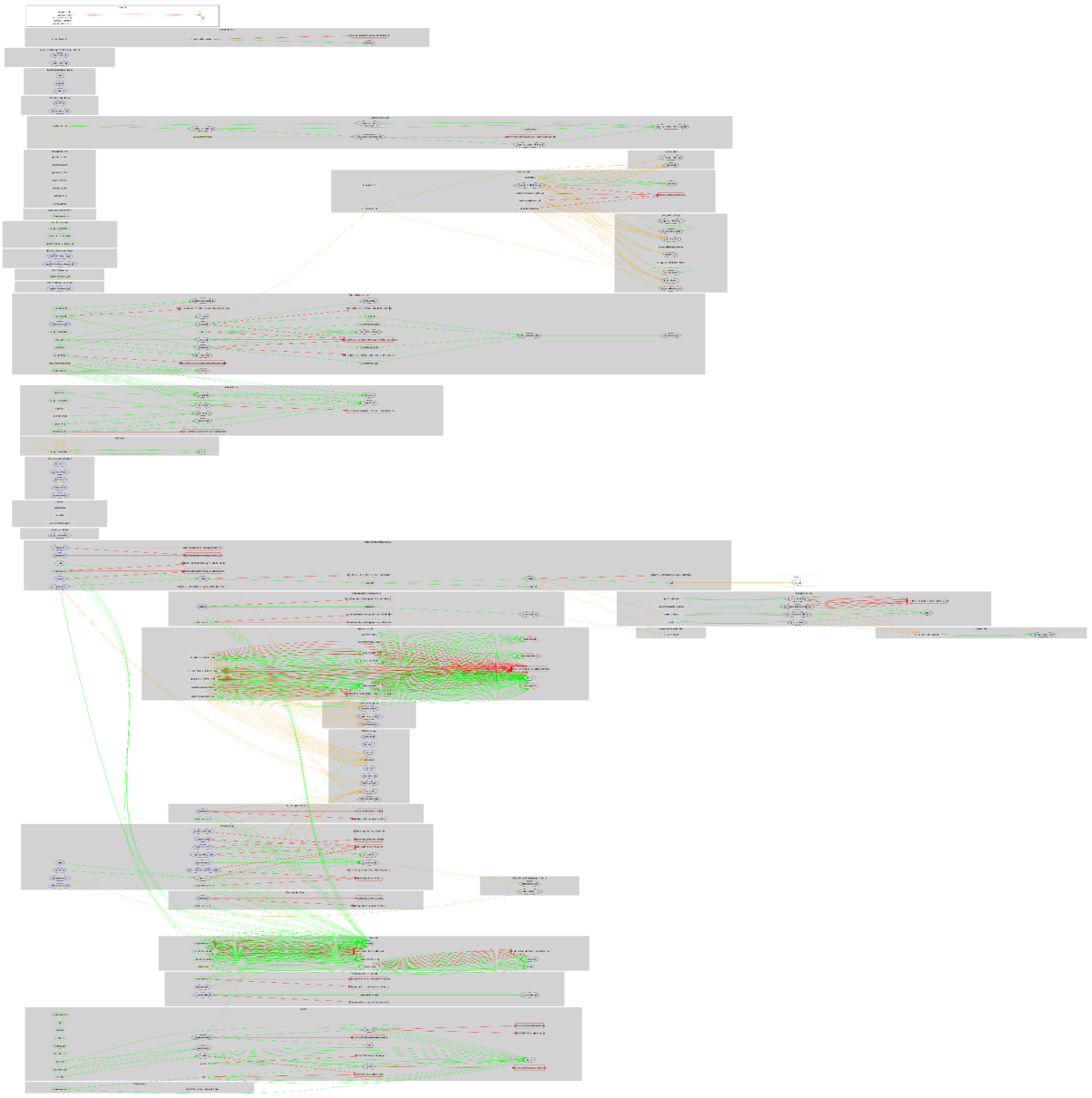
		Public	✓	Ownable
	setController	External	✓	onlyOwner
	vestedAmount	Public		-
	availableToRelease	Public		-
	releaseAvailable	External	✓	-
	_safeApprove	Private	✓	
WakeBeneficiaryVestingVault	Implementation	Ownable		
		Public	✓	Ownable
	releasable	Public		-
	vestedAmount	Public		-
	release	External	✓	-
ERC20	Implementation	Context, IERC20, IERC20Metadata, IERC20Errors		
		Public	✓	-
	name	Public		-
	symbol	Public		-
	decimals	Public		-
	totalSupply	Public		-
	balanceOf	Public		-
	transfer	Public	✓	-
	allowance	Public		-

	approve	Public	✓	-
	transferFrom	Public	✓	-
	_transfer	Internal	✓	
	_update	Internal	✓	
	_mint	Internal	✓	
	_burn	Internal	✓	
	_approve	Internal	✓	
	_approve	Internal	✓	
	_spendAllowance	Internal	✓	
WAKEToken	Implementation	ERC20		
		Public	✓	ERC20

Inheritance Graph



Flow Graph



Summary

The WAKE ecosystem implements a comprehensive token distribution and staking infrastructure. This audit investigates security issues, business logic concerns and potential improvements. WAKE is an interesting project that has a friendly and growing community. The Smart Contract analysis reported no compiler errors. The contract Owner can access some admin functions that should be handled with caution, as certain configurations could affect staking operations.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io